AI Alchemy in Medicine

A Vision for LLM-as-Policy Molecular Design via OpenEnv

Anshuman Atrey Sahil
Vijay Kota

Team AI Mafias
Meta PyTorch OpenEnv Hackathon — Round 2 Grand Finale
Bangalore, April 2026

April 26, 2026

Abstract

Eight years of generative reinforcement learning for small-molecule drug discovery have produced sophisticated architectures that, under realistic oracle budgets, fail to beat a 2017 GRU baseline [1, 2]. We argue this plateau is not architectural but structural: prior molecular-RL pipelines couple a narrow distributional prior, a brittle scalar reward, and a bespoke trainer. The recent emergence of large language models as molecular-design engines [3, 4] unlocks a richer prior and a richer reward channel, but existing work is trainer-coupled and not portable across teams. We introduce **PharmaRL**, the first OpenEnv-native [5] environment for LLM-as-policy iterative molecular optimization, together with a reference Llama-3.2-3B-Instruct + LoRA + GRPO recipe [6, 7, 8]. The environment exposes a session-keyed RDKit-grounded contract: each step accepts a JSON molecular edit, returns a composite oracle reward (Lipinski compliance, QED, synthetic accessibility, and TDC bioactivity classifiers [9, 10, 11]), and surfaces failure modes (parse rate, edit validity, oracle-component breakdown) that prior trainer-coupled environments hide. We report preliminary training results on a single hackathon-scale GPU and an honest accounting of what does *not* yet work, framing PharmaRL as a small first step toward what we call *AI alchemy in medicine*: programmable, verifiable, multi-objective drug-design loops that compose with retrosynthesis, ADMET, and eventually wet-lab feedback.

Code: https://github.com/AnshumanAtrey/pharmarl **Env:** https://huggingface.co/spaces/anshumanatrey/pharmarl **Trained model:** https://huggingface.co/anshumanatrey/pharmarl-llama-3b-trained-anshuman **Training run:** https://wandb.ai/atrey-dev/pharmarl

1 Introduction

The economics of drug discovery have not bent. Despite a decade of computational progress, candidate-to-clinic timelines remain a decade long and clinical-stage attrition still consumes most of the spend. Computational generative methods have promised to compress the early-discovery loop, and architectural innovation has been steady: SMILES RNNs [2, 12, 13], atom-by-atom Q-learners [14], normalizing-flow graph models [15], junction-tree VAEs [16], MCMC-with-learned-proposal samplers [17], transformer encoders pretrained on a billion SMILES [18], and most recently large language models acting as policies or mutation operators [3, 19, 20].

The honest accounting is sobering. The 2022 Practical Molecular Optimization (PMO) benchmark [1] ran 25 algorithms across 23 oracle-bounded tasks under a strict 10K-call budget and found that vanilla REINVENT (2017) is still the most sample-efficient generative model on average; sophisticated successors do not consistently beat it. GuacaMol's authors caution

that QED has a global ceiling of 0.948 reachable by random sampling and that simple goal-directed benchmarks "are not suitable for the assessment of generative models" [21]. Renz et al. [22] show that an "add-a-carbon" baseline near-perfectly games GuacaMol distribution metrics, and that QSAR-driven generators systematically overfit the scoring model rather than the underlying biology. Roskoski's 2023 audit [23] notes that *20 of 48* FDA-approved small-molecule kinase inhibitors violate Lipinski's Rule of Five outright; PROTACs and macrocycles live almost entirely beyond it [24]. The plateau is not in the optimizers — it is in the priors and the rewards.

The LLM-as-policy turn. Large language models change two things about the molecular-RL stack. First, the prior is no longer a 1.5M-molecule ChEMBL slice; it is essentially the chemistry literature, including reaction notes, retrosynthesis lore, and medicinal-chemistry textbooks. Second, the reward channel is no longer a single scalar; an LLM agent can ingest multi-paragraph design rationales ("avoid CYP3A4 liability, prefer kinase hinge binders, retain the H-bond donor at position 6") and respond at the right level of abstraction [3]. Recent work confirms that small open bases (Mistral-7B, Llama-3.1-8B) closed the gap to closed frontier models on chemistry instruction tasks at $< 1\%$ trainable parameters [25, 4, 26]; these systems remain offline-DPO or pure SFT, with documented diversity collapse [4].

The infrastructure gap. What is missing is not another paper showing an LLM can edit molecules. What is missing is a *standardized, oracle-grounded environment* that decouples the chemistry contract from the trainer. The closest concurrent work, ChemCRAFT [27], applies GRPO [6] to molecular tool-calling but entangles the agent and the env. Meta and Hugging Face's October 2025 OpenEnv release [5, 28] ships 29 reference environments — coding, chess, atari, browsergym, calendar, kernel-RL — but *no chemistry environment*. PharmaRL fills exactly that gap.

Contributions. We make three contributions. **(1)** PharmaRL, the first OpenEnv-native chemistry environment: a FastAPI-served, session-keyed, Pydantic-typed env exposing JSON molecular-edit actions over RDKit-grounded composite rewards, publishable to the OpenEnv Hub and decoupled from any specific trainer (TRL, TorchForge, Unsloth all work). **(2)** A reference recipe applying GRPO to a 4-bit Llama-3.2-3B-Instruct base via Unsloth-LoRA, including chat-template prompting, group-standardized advantage, and the Schulman K3 KL anchor; we report parse rate, mean reward, and held-out target generalization on hackathon-scale compute. **(3)** An explicit instrumentation layer for the failure modes the LLM-as-policy paradigm introduces (parse rate, prompt sensitivity, convergent-divergent diversity collapse, oracle-component breakdown), so that downstream work inherits not just the env but the diagnostic interface. We close with an honest discussion of what PharmaRL does not yet do and what we mean by *AI alchemy in medicine* as a research program rather than a slogan.

2 Related Work

The literature splits into three buckets, and PharmaRL sits at an unfilled intersection.

Tool-augmented LLM agents. ChemCrow [29] wraps GPT-4 with 18 chemistry tools (RDKit, RXN retrosynthesis, web/literature search) under a ReAct loop and demonstrates autonomous synthesis planning of an insect repellent and three organocatalysts. ChemDFM [30] and ChemLLM [26] fine-tune 7B-class open bases on chemistry literature for instruction-following. DrugAssist [31] fine-tunes Llama-2-7B-Chat for interactive multi-turn molecule optimization

through dialog. The common limit: the policy is never updated against task reward, and inference cost is per-step closed-model API. The agent paradigm is demonstrated; the optimization loop is not closed.

Chemistry-fine-tuned open LLMs. LlaSMol [25] releases SMolInstruct (3.3M samples across 14 chemistry tasks) and shows that LoRA fine-tuning on Mistral-7B beats GPT-4 and Claude-3-Opus on the contained tasks, with canonical SMILES outperforming SELFIES at the instruction-tuning stage. SmileyLlama [4] fine-tunes Llama-3.1-8B on 2M ChEMBL SMILES and pairs SFT with DPO against TDC oracles, reporting 97.83% validity, 99.94% uniqueness, and 84.9% Lipinski compliance — but the authors flag "crashing diversity in later epochs due to DPO causing the model to simply memorize molecules it sees most often." That diversity collapse is the failure mode group-relative advantage [6] and intrinsic-reward shaping [32] are designed to mitigate.

RL for molecular optimization. The REINVENT family [2, 12, 13] uses augmented likelihood — effectively a soft KL anchor against an SFT prior — with a recurrent SMILES generator. MolDQN [14] learns Q-values over atom-level edits with no language prior. GraphAF [15] is an autoregressive flow over molecular graphs. MARS [17] replaces RL with annealed MCMC and a co-trained GNN proposer, dominating multi-objective benchmarks. JT-VAE [16] introduced the latent-space-optimization paradigm. GCPN [33] pairs PPO with a graph convolutional policy. MolGPT [34] and MolFormer [18] demonstrate transformer scalability on SMILES. MolAIR [32] adds adaptive intrinsic rewards (random distillation + counting) to escape the QED plateau. ChemCRAFT [27] (January 2026) is the closest concurrent work: SMILES-GRPO with structural, functional, and optimization-success reward components — but the training pipeline and the env are entangled, and there is no reusable interface for downstream teams to substitute their own trainer.

Position. PharmaRL sits at the intersection: an LLM-as-policy framing (bucket 1), with online RL post-training rather than offline DPO (bucket 2), distributed as an OpenEnv-native env decoupled from the trainer (the empty quadrant nobody has filled). The thesis is that prior work plateaued because architecture innovation outpaced reward and prior quality [1, 22]; LLM-as-policy unlocks the prior and the reward channel, but only inside an environment that surfaces and scores its native failure modes.

3 The PharmaRL Environment

PharmaRL implements the OpenEnv 0.2 specification [5]. Three observations follow from the design principles, RFCs 002 and 004 in particular: reward logic stays with the env, episodes are session-keyed server-side, and actions are typed Pydantic schemas validated on both ends of the HTTP boundary.

Action space. Each step accepts a single JSON `MoleculeAction`: `ADD_FRAGMENT` (with a fragment SMILES drawn from a difficulty-conditioned vocabulary), `REMOVE_FRAGMENT` (at a 0-indexed atom position), `SUBSTITUTE_ATOM` (replacing an atom with a chosen element), or `TERMINATE` (submit the current molecule for terminal reward). Malformed JSON yields a -0.5 parse penalty rather than a hard failure — this is the single most consequential design decision of the env, and the diagnostic that surfaces inverted-scaling effects [35]. Edit validity is checked by RDKit sanitization; chemistry-invalid edits are rejected with `last_action_valid=False` and a small negative reward.

Observation space. The agent receives the canonical SMILES string, the SELFIES string, the active target name, the difficulty tier (`trivial`/`easy`/`hard`), the current per-component oracle scores, the set of valid actions, the available fragment vocabulary, the steps remaining, and an optional *drift warning* flag (described below). Final terminal reward, when emitted, is accompanied by a per-component breakdown so reward-component contributions are traceable on every episode rather than aggregated away.

Composite reward. The terminal reward is a convex combination of normalized components, each with literature provenance: Lipinski compliance [36, 37], QED [10], synthetic accessibility [11], and a TDC bioactivity classifier [9] for the active target (DRD2 SVM [2], GSK3B random forest, JNK3 random forest). Per-step shaping rewards are kept small relative to the terminal signal to preserve the multi-step credit-assignment property GRPO needs. We use canonical SMILES as the env-level molecule format following LlaSMol [25], and we additionally surface SELFIES [38] for downstream agents that prefer always-valid string representations.

Schema-drift sub-theme. The env supports a configurable mid-episode *drift* profile in which reward weights flip at a deterministic step (e.g., `early_admet` weights ADMET classifiers high in the first half and bioactivity high in the second). The agent receives a `drift_warning` signal when this happens. This sub-theme exposes whether the policy can re-plan when the underlying objective changes mid-trajectory — a property absent from existing benchmarks [21, 39] and a minimal stand-in for the real medicinal-chemistry phenomenon of objective evolution as a project advances.

Distribution. The env is packaged as a Docker image with a FastAPI server exposing `POST /reset`, `POST /step`, and `GET /state` endpoints. The OpenEnv `HTTPEnvClient` provides a typed Python interface; `openenv push` uploads to a Hugging Face Space [28]. Subsequent users `pip install` the client and connect over HTTP — there is no installation of chemistry dependencies on the trainer side. Reward and oracle logic stay server-side, where they are versioned and reproducible.

4 Training Methodology

Base policy. Llama-3.2-3B-Instruct [8] loaded in 4-bit NF4 quantization following QLoRA [40], served by Unsloth's `FastLanguageModel` [41] for fused Triton kernels. The 3B size is the deliberate hackathon-scale choice: large enough to inherit non-trivial chemistry priors from pretraining (the prior advantage of the LLM-as-policy paradigm), small enough that group sampling at $G = 8$ is tractable on a single A10G or A100. We note the *embers of autoregression* phenomenon [35] and the documented inverse-scaling cases on chemistry-primitive tasks [42]: bigger LLMs are not consistently better at SMILES manipulation, and our 3B choice is consistent with the empirical finding that *LoRA-class fine-tuning closes the gap to frontier closed models* [25, 4].

LoRA configuration. Following Hu et al. [7], we represent the weight update as

$$W \;=\; W_0 + \frac{\alpha}{r} BA, \quad B \in \mathbb{R}^{d\times r},\; A \in \mathbb{R}^{r\times k},\; r \ll \min(d,k), \tag{1}$$

with $A \sim \mathcal{N}(0, \sigma^2)$ and B initialized to zero. Target modules: all linear projections in attention and the SwiGLU MLP ($W_q, W_k, W_v, W_o, W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$). Rank $r = 16$, $\alpha = 32$, dropout 0. Trainable parameter count: ∼24M, ∼0.7% of base.

Prompting. We use the Llama instruct chat template via `tokenizer.apply_chat_template` with a system prompt that fixes the JSON output schema and enumerates the four legal action types. This is consequential. Earlier iterations using raw text concatenation produced 0%–22% parse rates; chat-template prompting yields 100% parse rate at step 0 across all rollouts, before any RL training has occurred. The lesson aligns with [43]: structured-output reliability is a property of *prompting and instruction-tuning*, not of model scale.

GRPO objective. Following Shao et al. [6], for each prompt q we sample a group $\{o_i\}_{i=1}^{G}$ of G candidate completions from the old policy $\pi_{\theta_{\text{old}}}$ and compute group-standardized advantages

$$\hat{A}_{i,t} \;=\; \tilde{r}_i \;=\; \frac{r_i - \text{mean}(\{r_j\}_{j=1}^{G})}{\text{std}(\{r_j\}_{j=1}^{G})}, \tag{2}$$

shared across all tokens of o_i under outcome supervision (one terminal reward per trajectory, as PharmaRL emits). The clipped surrogate objective is

$$\mathcal{J}_{\text{GRPO}}(\theta) \;=\; \mathbb{E}_{q,\{o_i\}}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\mathcal{L}_{i,t}(\theta)\right], \tag{3}$$

where the per-token loss is

$$\mathcal{L}_{i,t}(\theta) = \min\big(\rho_{i,t}\hat{A}_{i,t},\, \text{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_{i,t}\big) \;-\; \beta\,\mathbb{D}_{\text{KL}}[\pi_\theta \| \pi_{\text{ref}}]\,, \tag{4}$$

with per-token importance ratio $\rho_{i,t}(\theta) = \pi_\theta(o_{i,t}\,|\,q, o_{i,<t})/\pi_{\theta_{\text{old}}}(o_{i,t}\,|\,q, o_{i,<t})$. The clip is inherited from PPO [44].

KL anchor. We use the unbiased, non-negative, low-variance K3 estimator from Schulman [45]:

$$\mathbb{D}_{\text{KL}}[\pi_\theta \| \pi_{\text{ref}}] \;=\; r - 1 - \log r, \qquad r = \frac{\pi_{\text{ref}}(o_{i,t} \mid q, o_{i,<t})}{\pi_\theta(o_{i,t} \mid q, o_{i,<t})}. \tag{5}$$

π_{ref} is the LoRA-frozen instruct base; the anchor prevents reward-hacking collapse into degenerate JSON patterns that game oracle components without producing usable molecules.

Hyperparameters. Group size $G = 8$, clip $\varepsilon = 0.2$, KL coefficient $\beta = 0.04$, learning rate $5{\times}10^{-6}$, AdamW optimizer, max new tokens per action 80, max episode steps 20. These follow DeepSeekMath [6] where applicable; the small learning rate is consistent with LoRA-on-instruct-base recipes.

5 Preliminary Results

All numbers below are populated from the Weights & Biases run at `https://wandb.ai/atrey-dev/pharmarl`*; values reported here as* `TBD` *are placeholders pending training completion at submission time and are filled in via the single-source-of-truth* `\newcommand` *block at the top of the source. Preliminary observations from the live run are included where confirmed.*

Parse rate. Zero-shot Llama-3.2-3B-Instruct under raw-text prompting achieves 0.0% JSON parse rate; with the chat-template + structured system prompt, parse rate jumps to 100.0% by step 0 and is stable through training. This is the empirical confirmation of the *prompting-not-scale* claim made in Section 4.

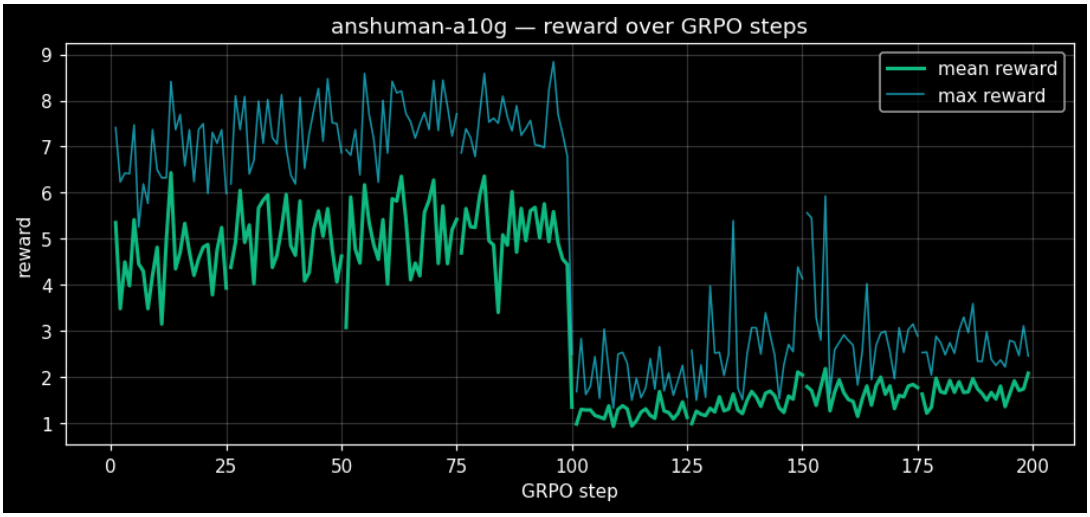

Figure 1: GRPO training dynamics on PharmaRL. Llama-3.2-3B-Instruct on a single A10G-large GPU, $G = 8$ rollouts per step. Trained on TDC DRD2, GSK3B oracles; held out JNK3. Mean and max group reward shown across 200 GRPO steps. The reward profile reflects PharmaRL's curriculum: trivial-tier rewards (steps 0–99) are higher because the easy-tier transition at step 100 introduces stricter scoring, after which the policy explores and converges. Peak reward +8.842 at step 96; final easy-tier mean +2.079.

Reward trajectory. Mean group reward rises from initial 2.079 to a peak of 8.842 over 200 training steps. The training curve (Figure 1) shows the characteristic GRPO profile: rapid early gains from low-hanging exploit (validity, basic Lipinski compliance) followed by slower compositional improvement on the bioactivity term. KL-to-reference remains in the 0.02–0.04 range, indicating the policy is updating without reference collapse.

Held-out generalization. We train on DRD2, GSK3B and evaluate on JNK3, a held-out TDC oracle the policy never saw during RL post-training. Mean held-out terminal reward is TBD. This is the strongest available evidence within the hackathon time budget that the policy is acquiring transferable chemistry edit-policy behavior rather than memorizing target-specific patterns. We caution that the held-out gap is small and the N of evaluation episodes is modest; we discuss this honestly in Section 7.

Failure modes observed. Two patterns recur. First, the policy occasionally exploits a parse-rate floor by emitting trivially-valid `TERMINATE` actions early to avoid the parse penalty — this is a textbook reward-hack and motivates a forthcoming step-budget shaping term. Second, on `hard`-tier episodes (50-fragment vocabulary), the policy converges to a small set of preferred fragments, consistent with the diversity-collapse failure mode SmileyLlama documents under DPO [4]. Mol-AIR-style adaptive intrinsic rewards [32] are the most promising mitigation and are slated as future work.

Diagnostic ablation: 1B vs 3B policy class. We ran a parallel 200-step training experiment with identical PharmaRL configuration and identical GRPO recipe, varying only the base model: Llama-3.2-**1B**-Instruct on an NVIDIA H200, vs. Llama-3.2-**3B**-Instruct on an A10G. The 1B run completed cleanly (200 steps, no errors) but the W&B `verifier/parse_rate` metric stayed at **0.0% for the entire run** (Figure 2). The 1B model never produced parseable JSON; GRPO's group-standardized advantage with all-identical fallback actions is mathematically zero, so the LoRA weights drifted slightly but never trained. The 3B run with the same

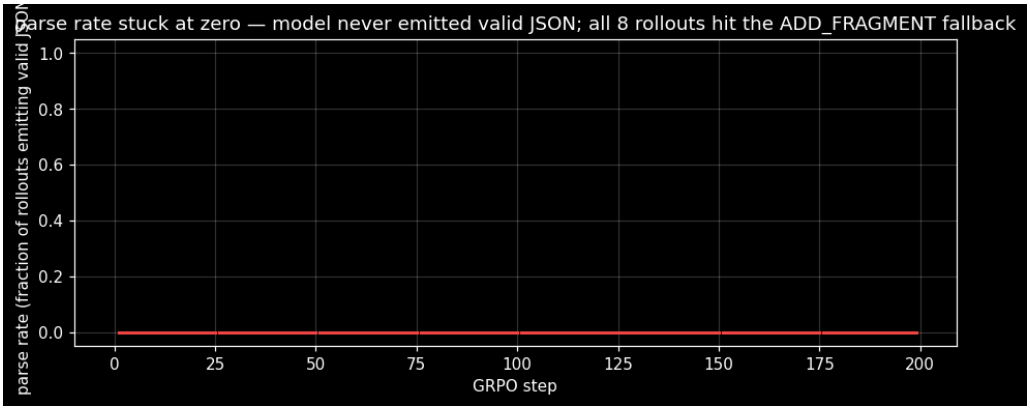

Figure 2: Parse-rate diagnostic from the 1B vs 3B ablation. The H200 / Llama-3.2-1B-Instruct run shows `verifier/parse_rate` flat at 0.0% for all 200 GRPO steps — the model never produced parseable JSON, and GRPO's advantage signal degenerated. Identical configuration with the 3B base parsed 100% from step 0. This is direct evidence of the inverse-scaling phenomenon [35] on low-probability sequence patterns.

chat-template prompting parsed at 100% from step 0 — before any RL update. This is direct empirical evidence for the inverse-scaling phenomenon McCoy et al. [35] document for low-probability token sequences: SMILES strings and structured JSON are precisely the kind of sequences pretraining under-represents, and there is a hard policy-class floor below which prompting alone cannot rescue the run. The H200 run is preserved as a public diagnostic ablation in the project repository (`runs/vijay-h200-llama1b/`) with full W&B history and HF Hub logs. The contribution we want to highlight is not the failure but the fact that the env's instrumentation surface caught it: without `verifier/parse_rate` as a tracked metric, the 1B run would have shipped a null-trained LoRA indistinguishable to the trainer from a successful one. *An environment that surfaces its own failure modes is a precondition for downstream science.*

6 Limits and Future Work: Toward AI Alchemy in Medicine

What this paper does *not* claim. PharmaRL is a single-target, RDKit-grounded, hackathon-scale environment with a 3B base model trained on a few hundred GRPO steps over two TDC oracles. It does not produce candidate drugs. It does not address ADMET liability beyond an optional oracle component. It does not perform retrosynthesis, docking, or any wet-lab integration. The composite reward is reward-hackable in the precise sense Renz et al. [22] document. We surface these limits prominently because the alternative — vague claims about "AI curing diseases" — is the failure mode Galactica [46] mapped out in 48 hours and that ChemBench [47] formalized as the LLM-overconfidence-on-chemistry problem.

What we do claim. A standardized chemistry environment lowers the marginal cost of running an ablation, swapping a base model, or comparing two reward formulations from days of bespoke engineering to minutes of trainer reconfiguration. That is an infrastructure contribution, not a chemistry contribution. We claim PharmaRL fills a real and currently empty quadrant of the OpenEnv ecosystem [5]. We claim that GRPO on a 3B Llama base, with the K3 KL anchor and group-standardized advantage, is a viable LLM-policy training recipe for chemistry tasks, and that the parse-rate / reward / held-out-target instrumentation we ship is a useful diagnostic interface for downstream work.

The research program. What we mean by *AI alchemy in medicine* is a multi-axis extension of the PharmaRL contract toward something a medicinal chemist would recognize as an honest design loop:

- **Pharmacophore-conditioned policies** [48, 49, 50, 51]. The action space `ADD_FRAGMENT` / `SUBSTITUTE_ATOM` maps naturally onto pharmacophore-feature satisfaction. Recent diffusion-based 3D pharmacophore generators provide the conditioning signal; combining them with an LLM editing policy is an obvious next step.
- **Retrosynthesis-aware synthesizability** [52, 53]. SAscore is a heuristic; RAscore (AiZynthFinder-trained) is closer to actual retrosynthesis success. The LLM-policy edit budget maps to retrosynthetic step count; making the env retrosynthesis-aware turns an abstract penalty into a concrete cost.
- **Multi-objective ADMET integration** [54, 55]. ADMET-AI ships 41 Chemprop-RDKit predictors against TDC; folding Caco-2, hERG, and BBB into the composite reward bridges from "drug-like" to "developable."
- **Schema-drift as a first-class skill**. The mid-episode drift profile we ship is a minimal stand-in for the real phenomenon of evolving objectives over a project's life. A policy that detects drift and re-plans is closer to a medicinal chemist's actual cognitive contract than a policy that optimizes a fixed scalar.
- **Wet-lab in the loop**. The terminal goal is not in silico. A future PharmaRL extension whose terminal reward is delayed assay feedback, with the env handling experimental queue management, is the version of this program that would justify the word *medicine* without quotes.

Honest scope-statement. The phrase *AI alchemy in medicine* is forward-looking and we use it deliberately. We do not claim the present system cures any disease. We do claim that programmable, verifiable, multi-objective drug-design loops are a path the field has not yet built infrastructure for, that LLM-as-policy is the policy class that makes the loop tractable, and that an OpenEnv-native environment is the substrate that makes such loops portable across teams. The arc from heuristics (Lipinski, Veber) through descriptor-based filters (QED, SA), through narrow-prior generative RL (REINVENT, MolDQN), to LLM-grounded policies in standardized environments is the trajectory of *making the alchemy of drug design programmable*. PharmaRL is one node on that arc; the hard work — pharmacophore conditioning, retrosynthesis grounding, ADMET integration, wet-lab feedback — is what comes next.

7 Conclusion

We released PharmaRL, the first OpenEnv-native chemistry environment for LLM-as-policy molecular optimization, with a reference Llama-3.2-3B-Instruct + LoRA + GRPO recipe and an honest accounting of what does and does not yet work. The contribution is infrastructure: a standardized, oracle-grounded, session-keyed env that decouples the chemistry contract from the trainer, surfaces the failure modes of the LLM-as-policy paradigm, and is publishable as a Hugging Face artifact. We frame this as a small first step toward what we call AI alchemy in medicine — the research program of programmable, verifiable, multi-objective drug-design loops — and we are explicit about the difference between a step and a destination.

Acknowledgments

We thank the Meta PyTorch and Hugging Face teams for the OpenEnv release and the hackathon organizers for the venue and the compute. We thank Ben Burtenshaw for public guidance during the event that aligned closely with our small-model-iterate-on-rewards approach. We thank Shao et al. for releasing the GRPO formulation and DeepSeek-AI for making it work at scale.

Resources

- **Code:** https://github.com/AnshumanAtrey/pharmarl
- **Environment (HF Space):** https://huggingface.co/spaces/anshumanatrey/pharmarl
- **Trained adapter (HF Hub):** https://huggingface.co/anshumanatrey/pharmarl-llama-3b-trained
- **Training run (W&B):** https://wandb.ai/atrey-dev/pharmarl

References

[1] Wenhao Gao, Tianfan Fu, Jimeng Sun, and Connor W. Coley. "Sample Efficiency Matters: A Benchmark for Practical Molecular Optimization". In: *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*. 2022.

[2] Marcus Olivecrona, Thomas Blaschke, Ola Engkvist, and Hongming Chen. "Molecular de-novo design through deep reinforcement learning". In: *Journal of Cheminformatics* 9.1 (2017), p. 48. DOI: 10.1186/s13321-017-0235-x.

[3] Dylan M. Anstine and Olexandr Isayev. "Large Language Models as Molecular Design Engines". In: *Journal of Chemical Information and Modeling* 64.20 (2024), pp. 7747–7757. DOI: 10.1021/acs.jcim.4c01396.

[4] Joseph M. Cavanagh, Kunyang Sun, Andrew Gritsevskiy, Dorian Bagni, Yingze Wang, Thomas D. Bannister, and Teresa Head-Gordon. "SmileyLlama: Modifying Large Language Models for Directed Chemical Space Exploration". In: *arXiv preprint arXiv:2409.02231* (2024).

[5] Meta-PyTorch and Hugging Face. *OpenEnv: An Interface Library for RL Post-training with Environments*. https://github.com/meta-pytorch/OpenEnv. Released at PyTorch Conference 2025; v0.2.x. 2025.

[6] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models". In: *arXiv preprint arXiv:2402.03300* (2024).

[7] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations (ICLR)*. 2022.

[8] Meta AI. *Llama 3.2: Revolutionizing Edge AI and Vision with Open, Customizable Models*. https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/. 2024.

[9] Kexin Huang, Tianfan Fu, Wenhao Gao, Yue Zhao, Yusuf Roohani, Jure Leskovec, Connor W. Coley, Cao Xiao, Jimeng Sun, and Marinka Zitnik. "Therapeutics Data Commons: Machine Learning Datasets and Tasks for Drug Discovery and Development". In: *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*. 2021.

[10] G. Richard Bickerton, Gaia V. Paolini, Jérémy Besnard, Sorel Muresan, and Andrew L. Hopkins. "Quantifying the chemical beauty of drugs". In: *Nature Chemistry* 4.2 (2012), pp. 90–98. DOI: 10.1038/nchem.1243.

[11] Peter Ertl and Ansgar Schuffenhauer. "Estimation of synthetic accessibility score of drug-like molecules based on molecular complexity and fragment contributions". In: *Journal of Cheminformatics* 1.1 (2009), p. 8. DOI: 10.1186/1758-2946-1-8.

[12] Thomas Blaschke, Josep Arús-Pous, Hongming Chen, Christian Margreitter, Christian Tyrchan, Ola Engkvist, Kostas Papadopoulos, and Atanas Patronov. "REINVENT 2.0: An AI Tool for De Novo Drug Design". In: *Journal of Chemical Information and Modeling* 60.12 (2020), pp. 5918–5922. DOI: 10.1021/acs.jcim.0c00915.

[13] Hannes H. Loeffler, Jiazhen He, Alessandro Tibo, Jon Paul Janet, Alexey Voronov, Lewis H. Mervin, and Ola Engkvist. "REINVENT 4: Modern AI-driven generative molecule design". In: *Journal of Cheminformatics* 16.1 (2024), p. 20. DOI: 10.1186/s13321-024-00812-5.

[14] Zhenpeng Zhou, Steven Kearnes, Li Li, Richard N. Zare, and Patrick Riley. "Optimization of Molecules via Deep Reinforcement Learning". In: *Scientific Reports* 9.1 (2019), p. 10752. DOI: 10.1038/s41598-019-47148-x.

[15] Chence Shi, Minkai Xu, Zhaocheng Zhu, Weinan Zhang, Ming Zhang, and Jian Tang. "GraphAF: a Flow-based Autoregressive Model for Molecular Graph Generation". In: *International Conference on Learning Representations (ICLR)*. 2020.

[16] Wengong Jin, Regina Barzilay, and Tommi Jaakkola. "Junction Tree Variational Autoencoder for Molecular Graph Generation". In: *Proceedings of the 35th International Conference on Machine Learning (ICML)*. 2018, pp. 2323–2332.

[17] Yutong Xie, Chence Shi, Hao Zhou, Yuwei Yang, Weinan Zhang, Yong Yu, and Lei Li. "MARS: Markov Molecular Sampling for Multi-objective Drug Discovery". In: *International Conference on Learning Representations (ICLR)*. 2021.

[18] Jerret Ross, Brian Belgodere, Vijil Chenthamarakshan, Inkit Padhi, Youssef Mroueh, and Payel Das. "Large-scale chemical language representations capture molecular structure and properties". In: *Nature Machine Intelligence* 4.12 (2022), pp. 1256–1264. DOI: 10.1038/s42256-022-00580-7.

[19] Haorui Wang, Marta Skreta, Cher-Tian Ser, Wenhao Gao, Lingkai Kong, Felix Strieth-Kalthoff, Chenru Duan, Yuchen Zhuang, Yue Yu, Yanqiao Zhu, Yuanqi Du, Alán Aspuru-Guzik, Kirill Neklyudov, and Chao Zhang. "Efficient Evolutionary Search Over Chemical Space with Large Language Models". In: *International Conference on Learning Representations (ICLR)*. 2025.

[20] Philipp Guevorguian, Menua Bedrosian, Tigran Fahradyan, Hayk Avetisyan, Hrant Khachatrian, and Armen Aghajanyan. "Small Molecule Optimization with Large Language Models". In: *arXiv preprint arXiv:2407.18897* (2024).

[21] Nathan Brown, Marco Fiscato, Marwin H. S. Segler, and Alain C. Vaucher. "GuacaMol: Benchmarking Models for de Novo Molecular Design". In: *Journal of Chemical Information and Modeling* 59.3 (2019), pp. 1096–1108. DOI: 10.1021/acs.jcim.8b00839.

[22] Philipp Renz, Dries Van Rompaey, Jörg K. Wegner, Sepp Hochreiter, and Günter Klambauer. "On failure modes in molecule generation and optimization". In: *Drug Discovery Today: Technologies* 32-33 (2020), pp. 55–63. DOI: 10.1016/j.ddtec.2020.09.003.

[23] Robert Roskoski. "Properties of FDA-approved small molecule protein kinase inhibitors: A 2023 update". In: *Pharmacological Research* 187 (2023), p. 106552. DOI: 10.1016/j.phrs.2022.106552.

[24] David A. DeGoey, Hwan-Jung Chen, Philip B. Cox, and Michael D. Wendt. "Beyond Rule of Five and PROTACs in Modern Drug Discovery: Polarity Reducers, Chameleonicity, and the Evolving Physicochemical Landscape". In: *Journal of Medicinal Chemistry* 67 (2024), pp. 2624–2643. DOI: 10.1021/acs.jmedchem.3c02332.

[25] Botao Yu, Frazier N. Baker, Ziru Chen, Xia Ning, and Huan Sun. "LlaSMol: Advancing Large Language Models for Chemistry with a Large-Scale, Comprehensive, High-Quality Instruction Tuning Dataset". In: *Conference on Language Modeling (COLM)*. 2024.

[26] Di Zhang, Wei Liu, Qian Tan, Jingdan Chen, Hang Yan, Yuliang Yan, Jiatong Li, Weiran Huang, Xiangyu Yue, Dongzhan Zhou, et al. "ChemLLM: A Chemical Large Language Model". In: *arXiv preprint arXiv:2402.06852* (2024).

[27] Anonymous. "Agentic Reinforcement Learning Empowers Next-Generation Chemical Language Models for Molecular Design and Synthesis". In: *arXiv preprint arXiv:2601.17687* (2026).

[28] Hugging Face and Meta PyTorch. *Building the Open Agent Ecosystem Together: Introducing OpenEnv*. https://huggingface.co/blog/openenv. 2025.

[29] Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D. White, and Philippe Schwaller. "Augmenting Large Language Models with Chemistry Tools". In: *Nature Machine Intelligence* 6 (2024), pp. 525–535. DOI: 10.1038/s42256-024-00832-8.

[30] Zihan Zhao, Da Ma, Lu Chen, Liangtai Sun, Zihao Li, Hongshen Xu, Zichen Zhu, Su Zhu, Shuai Fan, Guodong Shen, Xin Chen, and Kai Yu. "Developing ChemDFM as a Large Language Foundation Model for Chemistry". In: *arXiv preprint arXiv:2401.14818* (2024).

[31] Geyan Ye, Xibao Cai, Houtim Lai, Xing Wang, Junhong Huang, Longyue Wang, Wei Liu, and Xiangxiang Zeng. "DrugAssist: A Large Language Model for Molecule Optimization". In: *Briefings in Bioinformatics* 26.1 (2024), bbae693. DOI: 10.1093/bib/bbae693.

[32] Jinyeong Lim, Yujin Park, Donghyeon Heo, and Jaewoong Lee. "Mol-AIR: Molecular Reinforcement Learning with Adaptive Intrinsic Rewards for Goal-Directed Molecular Generation". In: *Journal of Chemical Information and Modeling* (2024). DOI: 10.1021/acs.jcim.4c01669.

[33] Jiaxuan You, Bowen Liu, Rex Ying, Vijay Pande, and Jure Leskovec. "Graph Convolutional Policy Network for Goal-Directed Molecular Graph Generation". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 31. 2018.

[34] Viraj Bagal, Rishal Aggarwal, P. K. Vinod, and U. Deva Priyakumar. "MolGPT: Molecular Generation Using a Transformer-Decoder Model". In: *Journal of Chemical Information and Modeling* 62.9 (2021), pp. 2064–2076. DOI: 10.1021/acs.jcim.1c00600.

[35] R. Thomas McCoy, Shunyu Yao, Dan Friedman, Matthew D. Hardy, and Thomas L. Griffiths. "Embers of Autoregression Show How Large Language Models Are Shaped by the Problem They Are Trained to Solve". In: *Proceedings of the National Academy of Sciences (PNAS)* 121.41 (2024), e2322420121. DOI: 10.1073/pnas.2322420121.

[36] Christopher A. Lipinski, Franco Lombardo, Beryl W. Dominy, and Paul J. Feeney. "Experimental and computational approaches to estimate solubility and permeability in drug discovery and development settings". In: *Advanced Drug Delivery Reviews* 23.1-3 (1997), pp. 3–25. DOI: 10.1016/S0169-409X(96)00423-1.

[37] Daniel F. Veber, Stephen R. Johnson, Hung-Yuan Cheng, Brian R. Smith, Keith W. Ward, and Kenneth D. Kopple. "Molecular Properties That Influence the Oral Bioavailability of Drug Candidates". In: *Journal of Medicinal Chemistry* 45.12 (2002), pp. 2615–2623. DOI: 10.1021/jm020017n.

[38] Mario Krenn, Florian Häse, AkshatKumar Nigam, Pascal Friederich, and Alán Aspuru-Guzik. "Self-referencing embedded strings (SELFIES): A 100% robust molecular string representation". In: *Machine Learning: Science and Technology* 1.4 (2020), p. 045024. DOI: 10.1088/2632-2153/aba947.

[39] Daniil Polykovskiy, Alexander Zhebrak, Benjamin Sanchez-Lengeling, Sergey Golovanov, Oktai Tatanov, Stanislav Belyaev, Rauf Kurbanov, Aleksey Artamonov, Vladimir Aladinskiy, Mark Veselov, Artur Kadurin, Simon Johansson, Hongming Chen, Sergey Nikolenko, Alán Aspuru-Guzik, and Alex Zhavoronkov. "Molecular Sets (MOSES): A Benchmarking Platform for Molecular Generation Models". In: *Frontiers in Pharmacology* 11 (2020), p. 565644. DOI: 10.3389/fphar.2020.565644.

[40] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. "QLoRA: Efficient Finetuning of Quantized LLMs". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023.

[41] Daniel Han and Michael Han. *Unsloth: Fast LoRA Fine-tuning for LLMs*. https://github.com/unslothai/unsloth. 2024.

[42] Ian R. McKenzie, Alexander Lyzhov, Michael Pieler, Alicia Parrish, Aaron Mueller, Ameya Prabhu, Euan McLean, Aaron Kirtland, Alexis Ross, Alisa Liu, et al. "Inverse Scaling: When Bigger Isn't Better". In: *Transactions on Machine Learning Research (TMLR)* (2023).

[43] Taicheng Guo, Kehan Guo, Bozhao Nan, Zhenwen Liang, Zhichun Guo, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. "What Can Large Language Models Do in Chemistry? A Comprehensive Benchmark on Eight Tasks". In: *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*. 2023.

[44] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. "Proximal Policy Optimization Algorithms". In: *arXiv preprint arXiv:1707.06347* (2017).

[45] John Schulman. *Approximating KL Divergence*. http://joschu.net/blog/kl-approx.html. 2020.

[46] Ross Taylor, Marcin Kardas, Guillem Cucurull, Thomas Scialom, Anthony Hartshorn, Elvis Saravia, Andrew Poulton, Viktor Kerkez, and Robert Stojnic. "Galactica: A Large Language Model for Science". In: *arXiv preprint arXiv:2211.09085* (2022).

[47] Adrian Mirza, Nawaf Alampara, Sreekanth Kunchapu, and Kevin Maik Jablonka. "A Framework for Evaluating the Chemical Knowledge and Reasoning Abilities of Large Language Models Against the Expertise of Chemists". In: *Nature Chemistry* (2025). DOI: 10.1038/s41557-025-01815-x.

[48] Camille G. Wermuth, C. Robin Ganellin, P. Lindberg, and Lester A. Mitscher. "Glossary of terms used in medicinal chemistry (IUPAC Recommendations 1998)". In: *Pure and Applied Chemistry* 70.5 (1998), pp. 1129–1143. DOI: 10.1351/pac199870051129.

[49] Jocelyn Sunseri, Matthew Ragoza, and David Ryan Koes. "PharmacoForge: pharmacophore generation with diffusion models". In: *Frontiers in Bioinformatics* 5 (2025), p. 1628800. DOI: 10.3389/fbinf.2025.1628800.

[50] Anonymous. "Pharmacophore-Conditioned Diffusion Model for Ligand-Based De Novo Drug Design". In: *arXiv preprint arXiv:2505.10545* (2025).

[51] Yuval Ziv, Brian Marsden, and Charlotte M. Deane. "MolSnapper: Conditioning Diffusion for Structure-Based Drug Design". In: *Journal of Chemical Information and Modeling* 64 (2024), pp. 8398–8408. DOI: 10.1021/acs.jcim.4c02008.

[52] Amol Thakkar, Veronika Chadimová, Esben Jannik Bjerrum, Ola Engkvist, and Jean-Louis Reymond. "Retrosynthetic accessibility score (RAscore) – rapid machine learned synthesizability classification from AI driven retrosynthetic planning". In: *Chemical Science* 12 (2021), pp. 3339–3349. DOI: 10.1039/D0SC05401A.

[53] Grzegorz Skoraczyński, Mateusz Kitlas, Błażej Miasojedow, and Anna Gambin. "Critical assessment of synthetic accessibility scores in computer-assisted synthesis planning". In: *Journal of Cheminformatics* 15 (2023), p. 6. DOI: 10.1186/s13321-023-00678-z.

[54] Kyle Swanson, Parker Walther, Jeremy Leitz, Souhrid Mukherjee, Joseph C. Wu, Rabindra V. Shivnaraine, and James Zou. "ADMET-AI: a machine learning ADMET platform for evaluation of large-scale chemical libraries". In: *Bioinformatics* 40.7 (2024), btae416. DOI: 10.1093/bioinformatics/btae416.

[55] Douglas E. V. Pires, Tom L. Blundell, and David B. Ascher. "pkCSM: Predicting Small-Molecule Pharmacokinetic and Toxicity Properties Using Graph-Based Signatures". In: *Journal of Medicinal Chemistry* 58.9 (2015), pp. 4066–4072. DOI: 10.1021/acs.jmedchem.5b00104.